Karlsruher Institut für Technologie

Institut für Programmstrukturen und Datenorganisation

Prof. Dr.-Ing. Gregor Snelting
gregor.snelting@kit.edu

Prof. Dr. Ralf H. Reussner
reussner@kit.edu

Klausur Programmierparadigmen — Beispiellösung

WS22/23, 31. März 2023, 11:30 – 13:30 Uhr

Zugelassene Hilfsmittel:
Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

Bearbeitungszeit: 120 min

Aufgabe 1 (Haskell: neuronale Netze) [18 Punkte]

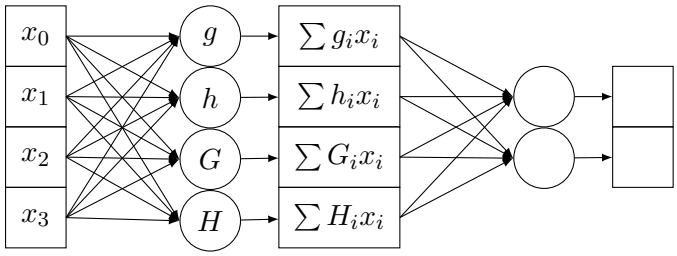
In dieser Aufgabe geht um neuronale Netze. Diese bestehen aus *Schichten* von *Neuronen*.

Ein Neuron hat n Gewichte g_i, bekommt n Eingaben x_i, und berechnet daraus seine Ausgabe als $\sum_{i=0}^{n-1} g_i x_i$. Die Neuronen einer Schicht bekommen alle die gleiche Eingabe und erzeugen daraus parallel ihre Ausgaben. Diese Werte sind die Eingabe für die Neuronen der nächsten Schicht.

Wir stellen Neuronen als Liste ihrer Gewichte dar, Schichten als Liste von Neuronen, und neuronale Netze als Liste von Schichten. Sie können davon ausgehen, dass alle Listen passende Längen haben.

```
type Neuron = [Double]
type Layer = [Neuron]
type Net = [Layer]

type Input = [Double]
```



(a) Implementieren Sie eine Funktion `evalN :: Input -> Neuron -> Double`, die die Ausgabe eines Neurons berechnet. [2 Punkte]

(b) Implementieren Sie eine Funktion `evalL :: Input -> Layer -> Input`, die die Ausgabe einer Schicht berechnet. [2 Punkte]

(c) Implementieren Sie eine Funktion `eval :: Input -> Net -> Input`, die die Ausgabe eines neuronalen Netzes berechnet. [2 Punkte]
Das erste Argument von `eval` ist die Eingabe der ersten Schicht, das Ergebnis ist die Ausgabe der letzten Schicht.

Um neuronale Netze zu trainieren, kann man z. B. einzelne Gewichte leicht verändern („perturbieren“) und überprüfen, welche Änderung dem Ziel am nächsten kommt. Um dies zu implementieren, brauchen wir zuerst einige Hilfsfunktionen.

Im Folgenden dürfen Sie die Funktion `setAt :: Int -> a -> [a] -> [a]` benutzen. Der Aufruf `setAt i x list` gibt `list` zurück, wobei das `i`-te Element auf `x` gesetzt ist.

(d) Implementieren Sie eine Funktion [2 Punkte]
`modifyAt :: Int -> (a -> a) -> [a] -> [a]`
Der Aufruf `modifyAt i f list` soll `list` zurückgeben, wobei auf das `i`-te Element `f` angewendet wurde.

(e) Implementieren Sie eine Funktion [2 Punkte]
`modifyAll :: (a -> a) -> [a] -> [[a]]`
Das Ergebnis von `modifyAll f list` ist eine Liste von Listen. Deren i-tes Element soll die Liste `list` sein, wobei auf das i-te Element `f` angewendet wurde.
Beispiel:
`modifyAll (+10) [1, 2, 3]` ⇒ `[[11, 2, 3], [1, 12, 3], [1, 2, 13]]`

(f) Implementieren Sie eine Funktion [3 Punkte]
`perturbNeuron :: Double -> Neuron -> [Neuron]`
Der Aufruf `perturbNeuron epsilon neuron` soll die Liste aller Neuronen zurückgeben, die sich von `neuron` in genau einem Gewicht um `+epsilon` oder `-epsilon` unterscheiden.
Hinweis:
Der Haskell-Ausdruck `(-epsilon)` ist *keine* Unterversorgung von `(-)`.

Ab jetzt sei auch `perturbNet :: Double -> Net -> [Net]` definiert.
`perturbNet epsilon net` gibt die Liste aller neuronaler Netze zurück, die sich von `net` in genau einem Gewicht eines Neurons um `+epsilon` oder `-epsilon` unterscheiden.

(g) Implementieren Sie nun eine Funktion [5 Punkte]
`ascend :: Double -> Input -> (Input -> Double) -> Net -> Net`.
Der Aufruf `ascend epsilon input goal net` soll einen Trainingsschritt für das neuronale Netz `net` durchführen. Dazu ist die Zielfunktion `goal` gegeben, die die Ausgabe des Netzes bei Eingabe von `input` bewertet. `ascend` soll die Perturbation von `net` mit der höchsten Bewertung finden.
Hinweis:
Sie können die berechnete Bewertung und die Perturbation, für die sie berechnet wurde, zu einem Paar zusammenfassen, und sich die lexikalische Ordnung auf Tupeln zunutze machen.

Beispiellösung:

```
evalN :: Input -> Neuron -> Double
evalN inputs weights = sum (zipWith (*) inputs weights)

evalL :: Input -> Layer -> Input
evalL inputs neurons = map (evalN inputs) neurons

eval :: Input -> Net -> Input
eval inputs layers = foldl evalL inputs layers

modifyAt i f xs = setAt i (f (xs !! i)) xs

modifyAll f xs = [ modifyAt i f xs | i <- [0 .. length xs - 1]]

perturbNeuron :: Double -> Neuron -> [Neuron]
perturbNeuron epsilon neuron = modifyAll (+ epsilon) neuron ++
                               modifyAll (subtract epsilon) neuron

ascend :: Double -> [Double] -> ([Double] -> Double) -> Net -> Net
ascend epsilon input goalF net = snd (maximum candidates)
    where
      candidates = map (\n -> (goalF (eval input n), n))
                       (perturbNet epsilon net)
```

Aufgabe 2 (Haskell: Unendliche Listen) [12 Punkte]

Gegeben sei der folgende Haskell-Datentyp für unendliche Listen:

```
data Infinite a = I a (Infinite a)
x # xs = I x xs
infixr 5 #
```

Den in Haskell üblichen Datentyp für Listen `[a]` bezeichnen wir fortan zur Unterscheidung als den Typ der *regulären* Listen. Im Gegensatz zum regulären Listen-Typ enthält `Infinite` keine endlichen Listen. Der Cons-Operator `(#)` ist, wie `(:)` auf regulären Listen, rechtsassoziativ und wird nur für die Beispiele unten als kurzer Alias für `I` benötigt.

(a) Implementieren Sie die Funktion `iota :: Int -> Infinite Int`, die alle natürlichen Zahlen beginnend beim übergebenen `Int` auflistet. [1 Punkt]
Beispiel: `iota 3 = 3 # 4 # 5 # ...`

(b) Geben Sie den allgemeinsten Haskell-Typ der Funktion [2 Punkte]
`tailI (I _ xs) = xs`
an. Inwiefern ist diese Funktion sicherer als `tail` auf regulären Listen? Begründen Sie anhand der Semantik des Ausdrucks `tail []`.

(c) Implementieren Sie die Funktion `takeI :: Int -> Infinite a -> [a]`. Wie auf regulären Listen soll `takeI n xs` die ersten `n` Elemente der unendlichen Liste zurückgeben. [2 Punkte]
Beispiel: `takeI 2 (iota 0) = [0,1]`

(d) Implementieren Sie die Funktion [1 Punkt]
`mapI :: (a -> b) -> Infinite a -> Infinite b`. Wie auf regulären Listen soll `mapI f xs` die Funktion `f` auf jedes Element von `xs` anwenden.
Beispiel: `mapI (+ 42) (iota 0) = 42 # 43 # 44 # ...`

(e) Implementieren Sie die Funktion [2 Punkte]
`prependI :: [a] -> Infinite a -> Infinite a`.
`prependI pref xs` soll die reguläre Liste `pref` der unendlichen Liste `xs` voranstellen.
Beispiel: `prependI [1,0] (iota 3) = 1 # 0 # 3 # 4 # 5 # ...`

(f) Implementieren Sie die Funktion [2 Punkte]
`intercalateI :: [a] -> Infinite a -> Infinite a`.
`intercalateI fill xs` soll zwischen jedem Element von `xs` die reguläre Liste `fill` einfügen.
Beispiel: `intercalateI [1,0] (iota 3) = 3 # 1 # 0 # 4 # 1 # 0 # 5 ...`

(g) Implementieren Sie die Funktion [2 Punkte]
`prefixesI :: Infinite a -> Infinite [a]`. `prefixesI xs` soll alle endlichen Präfixe von `xs` zurückgeben.
Beispiel: `prefixesI (iota 3) = [] # [3] # [3,4] # ...`

Beispiellösung: Zu (b): Allgemeinster Typ s.u.. Der Vorteil gegenüber `tail` ist, dass es auf jeder Eingabe definiert ist. Im Gegensatz dazu führt `tail []` zu einem Programmabsturz.

```
tailI :: Infinite a -> Infinite a

iota :: Int -> Infinite Int
iota n = I n (iota (n+1))

takeI :: Int -> Infinite a -> [a]
takeI n (I x xs)
  | n > 0     = x : takeI (n-1) xs
  | otherwise = []

mapI :: (a -> b) -> Infinite a -> Infinite b
mapI f (I x xs) = I (f x) (mapI f xs)

prependI :: [a] -> Infinite a -> Infinite a
prependI []     ys = ys
prependI (x:xs) ys = I x (prependI xs ys)

prependI2 :: [a] -> Infinite a -> Infinite a
prependI2 xs ys = foldr (#) ys xs

intercalateI :: [a] -> Infinite a -> Infinite a
intercalateI fill (I x xs) = I x (prependI fill (intercalateI fill xs))

prefixesI :: Infinite a -> Infinite [a]
prefixesI (I x xs) = I [] (mapI (x:) (prefixesI xs))

prefixesI2 :: Infinite a -> Infinite [a]
prefixesI2 xs = mapI (\n -> takeI n xs) (iota 0)

prefixesI3 :: Infinite a -> Infinite [a]
prefixesI3 xs = aux 0
  where
    aux n = takeI n xs # aux (n+1)
```

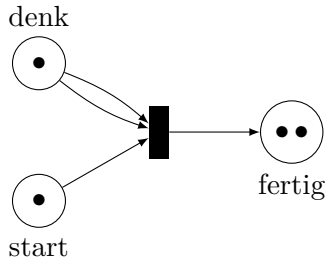
Aufgabe 3 (Prolog: Petrinetze) [16 Punkte]

Ein Petrinetz besteht aus einer Menge von *Stellen* und einer Menge von *Transitionen*. Auf jeder Stelle befindet sich eine Anzahl (≥ 0) an *Marken*.

Jede Transition hat eine Liste von Eingabe- und Ausgabestellen. Wenn die Transition *schaltet*, nimmt sie von jeder Eingabestelle eine Marke weg, und legt auf jeder Ausgabestelle eine Marke ab. Wenn eine Stelle mehrfach als Ein- oder Ausgabe auftaucht, nimmt die Transition entsprechend viele Marken weg oder legt sie ab. Die Transition kann nur schalten, wenn genügend Marken für die Eingabe vorhanden sind.

Wir modellieren Petrinetze folgendermaßen in Prolog:

- Der Zustand des Netzes ist eine Liste von Positionen von Marken. Z. B. repräsentiert `[start, denk, fertig, fertig]` eine Marke an den Stellen `start` und `denk` und zwei Marken an der Stelle `fertig`. Die Reihenfolge der Marken spielt keine Rolle.
- Eine Transition ist ein Paar von Listen. Die erste Liste sind die Eingabestellen, die zweite die Ausgabestellen. Z.B. nimmt die Transition `([start, denk, denk], [fertig])` beim Schalten eine Marke von `start`, zwei Marken von `denk`, und legt eine Marke auf `fertig` ab.
- Ein Petrinetz ist eine Liste von Transitionen.



(a) Implementieren Sie ein Prädikat `step(S, Net, T)`, das genau dann erfüllbar ist, wenn das Petrinetz `Net` durch Schalten einer Transition vom Zustand `S` in den Zustand `T` übergeht. [4 Punkte]
Hinweis: Bedenken Sie, dass die Marken in `S` und `T` nicht in der gleichen Reihenfolge stehen könnten.

Sie dürfen dafür das Prädikat `consume(L1, Xs, L2)` benutzen. Dieses ist erfüllbar, wenn die Liste `L2` aus `L1` entsteht, indem man ein Vorkommen jedes Elements der Liste `Xs` löscht. Sind in `L1` nicht genug Elemente vorhanden, ist das Prädikat nicht erfüllbar.
Beispiele:
`? consume([1, 1, 2, 2, 3], [1, 2, 1], X)` $\Rightarrow$ `X = [2, 3]`
`? consume([1, 1, 2, 2, 3], [1, 2, 4], X)` $\Rightarrow$ `False`

(b) Ein Petrinetz heißt *beschränkt*, wenn ein n existiert, sodass es niemals mehr als n Marken enthalten kann. Eine hinreichende Bedingung dafür ist, dass jede Transition nicht mehr Marken ablegt als sie wegnimmt. Implementieren Sie diese Bedingung im Prädikat `bounded(Net)`. [4 Punkte]

(c) Für den folgenden Teil brauchen wir zunächst ein Hilfsprädikat. Implementieren Sie ein Prädikat `pmember(L, LL)`, das genau dann erfüllbar ist, wenn eine Permutation der Liste `L` in der Liste von Listen `LL` vorkommt. [3 Punkte]

(d) Beschränkte Petrinetze können nur eine endliche Anzahl Zustände annehmen. Man kann also in endlicher Zeit die Erreichbarkeit eines Zustandes testen. Dies tut das folgende Prädikat: [5 Punkte]

```
reachable(Init, Net, T) :-
  iter(Init, Net, [Init], T1),
  permute(T1, T).
```

Implementieren Sie das Hilfsprädikat `iter(S, Net, Visited, T)`. Es soll genau dann erfüllbar sein, wenn vom Zustand `S` aus das beschränkte Petrinetz `Net` den Zustand `T` erreichen kann ohne dazwischen einen Zustand aus `Visited` zu besuchen. Zustände, die sich nur in der Reihenfolge der Marken unterscheiden, soll `iter` nicht mehrfach besuchen.

Hinweis: Um zu prüfen, ob ein Zustand schon besucht ist, bietet sich `pmember` an.

Beispiellösung:

```
step(Before, Net, After) :-
    member((In, Out), Net),
    consume(Before, In, T1),
    append(T1, Out, T2),
    permute(T2, After).

bounded([]).
bounded([(In, Out)|Net]) :-
    length(In, IL),
    length(Out, OL),
    IL >= OL,
    bounded(Net).

pmember(S, [Set|_]) :-
    permute(S, Set).
pmember(S, [_|Rest]) :-
    pmember(S, Rest).

iter(Init, _, _, Init).
iter(Init, Net, Visited, R) :-
    step(Init, Net, S),
    not(pmember(S, Visited)),
    iter(S, Net, [S|Visited], R).
```

Aufgabe 4 (Rationale Zahlen im λ-Kalkül) [15 Punkte]

Die nichtnegativen rationalen Zahlen $\mathbb{Q}_{\geq 0}$ lassen sich im λ-Kalkül als Paare von Church-Zahlen darstellen. Dabei wird eine rationale Zahl $\frac{p}{q}$ dargestellt als das Paar (c_p, c_q).

Hinweis: Das Verhalten bei Division durch 0, also (c_p, c_0), darf in dieser Aufgabe ignoriert werden.

Hierbei verwenden wir die aus der Übung bekannte Implementierung von Paaren, wir können also davon ausgehen, dass wir ein Paar von `p` und `q` durch den Ausdruck `pair p q` darstellen können, und dass wir die Komponenten eines Paares `r` durch `fst r` und `snd r` erhalten können, es gilt also

$$\texttt{fst (pair p q)} \Rightarrow^* \texttt{p} \text{ und}$$
$$\texttt{snd (pair p q)} \Rightarrow^* \texttt{q}.$$

Ebenfalls verwenden dürfen Sie alle aus der Vorlesung und Übung bekannten Operationen auf Church-Zahlen und Church-Booleans, insbesondere die Grundrechenarten `plus`, `minus` und `times` und Vergleichsoperatoren `eq` und `lt`, für welche gilt:

$$\texttt{plus}\ c_m\, c_n \Rightarrow^* c_{m+n},$$
$$\texttt{minus}\ c_m\, c_n \Rightarrow^* \begin{cases} c_{m-n} & \text{falls } m \geq n, \\ c_0 & \text{sonst,} \end{cases}$$
$$\texttt{times}\ c_m\, c_n \Rightarrow^* c_{mn},$$

$$\texttt{eq}\ c_m\, c_n \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } m = n, \\ c_{\text{false}} & \text{sonst,} \end{cases}$$
$$\texttt{lt}\ c_m\, c_n \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } m < n, \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

(a) Definieren Sie die Multiplikation `timesRat` zweier rationaler Zahlen. [3 Punkte]

(b) Definieren Sie die Addition `plusRat` zweier rationaler Zahlen. [3 Punkte]

(c) Definieren Sie eine Funktion `eqRat`, welche zwei rationale Zahlen auf Gleichheit testet, d.h. es soll gelten [4 Punkte]

$$\texttt{eqRat r s} \Rightarrow^* \begin{cases} c_{\text{true}} & \text{falls } \texttt{r} = \texttt{s}, \\ c_{\text{false}} & \text{sonst.} \end{cases}$$

Verwenden Sie hierzu den Gleichheitstest für Church-Zahlen `eq`.

(d) Definieren Sie die Funktion `floor`, welche eine rationale Zahl r auf die nächstkleinere oder gleiche Church-Zahl $c_{\lfloor r \rfloor}$ abbildet, also die Nachkommastellen von r abschneidet. [5 Punkte]
Hinweis: Der Rekursionsoperator `Y` könnte von Nutzen sein.
Beispiellösung:

```
-- Preliminaries
isZero = \n. n (\x. false) true
eq     = \m. \n. and (isZero (minus m n)) (isZero (minus n m))
lt     = \m. \n. isZero (minus (succ m) n)

-- (a)
timesRat = \r. \s.
  pair (times (fst r) (fst s)) (times (snd r) (snd s))

-- (b)
plusRat = \r. \s.
  pair (plus (times (fst r) (snd s)) (times (snd r) (fst s)))
```

```
    (times (snd r) (snd s))

-- (c)
eqRat = \r. \s.
  eq (times (fst r) (snd s)) (times (fst s) (snd r))

-- (d)
floor = Y (\floor. \r.
  (lt (fst r) (snd r)) c0
    (succ (floor (pair (minus (fst r) (snd r)) (snd r)))))
```

Aufgabe 5 (Unifikation) [8 Punkte]

Gegeben sei die einelementige Menge von Gleichungen $C = \{$

$$\begin{array}{rl} & f(f(X_1, g), X_2) \\ = & f(f(f(X_2, g), X_1), g) \end{array}$$

$\}$

über den Variablen X_1, X_2.

Führen Sie den Unifikationsalgorithmus nach Robinson zur Berechnung von `unify(`C`)` aus.
Geben Sie bei jedem rekursiven Aufruf von `unify` die erzeugte Substitution sowie die noch zu unifizierende Menge an. Geben Sie das Endergebnis, falls vorhanden, in Listenform an.

Zur Erinnerung: `unify`$(C) =$

```
if C == ∅ then []
else let {θl = θr} ⊎ C' = C in
  if θl == θr then unify(C')
  else if θl == Y and Y ∉ FV(θr) then unify([Y ⇨ θr] C') ∘ [Y ⇨ θr]
  else if θr == Y and Y ∉ FV(θl) then unify([Y ⇨ θl] C') ∘ [Y ⇨ θl]
  else if θl == f(θl^1,...,θl^n) and θr == f(θr^1,...,θr^n)
       then unify(C' ∪ {θl^1 = θr^1,...,θl^n = θr^n})
  else fail
```

Beispiellösung:

`unify`(C)
= `unify`$(\{f(X_1, g) = f(f(X_2, g), X_1), X_2 = g\})$
= `unify`$(\{X_2 = g, X_1 = f(X_2, g), g = X_1\})$
= `unify`$(\{X_1 = f(g, g), g = X_1\}) \circ [X_2 \Rightarrow g]$
= `unify`$(\{g = f(g, g)\}) \circ [X_1 \Rightarrow f(g, g)] \circ [X_2 \Rightarrow g]$
= **`fail`** (da auf linker und rechter Seite verschieden Nicht-Variablen)

C ist also nicht unifizierbar.

Durch geschickte Wahl von Gleichungen ist auch eine Ausführung in einem Schritt weniger möglich.

Aufgabe 6 (MPI) [15 Punkte]

Gegeben sei die untenstehende Methode `foo`, die als Parameter die Werte `x` und `root` entgegennimmt. Gehen Sie davon aus, dass MPI korrekt initialisiert und finalisiert wird. Die Methode `qsort` stammt aus der C-Standardbibliothek. Sie sortiert sequentiell ein Array `array` mit dem QuickSort-Algorithmus, wobei die Einträge mittels der übergebenen `cmpfunc` verglichen werden.

Die Vergleichsfunktion `cmpfunc` gibt -1 zurück, wenn der linke Parameter vor dem rechten Parameter einsortiert werden soll, 0, falls der linke und rechte Parameter gleichwertig sind, und 1, falls der rechte Parameter vor dem linken einsortiert werden soll.

Signatur `qsort`:

```
void qsort(void * array, size_t elementCount, size_t elementSize,
int (*cmpfunc)(const void *, const void *));
```

```
int cmpfunc (const void * a, const void * b) {
    return ( *(int*)a - *(int*)b );
}

int foo(int x, int root) {
    int size, rank, result;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    MPI_Barrier(MPI_COMM_WORLD);

    int all_numbers[size];

    MPI_Gather(&x, 1, MPI_INT, &all_numbers, 1, MPI_INT, root,
       MPI_COMM_WORLD);

    if (rank == root) qsort(&all_numbers, size, sizeof(int), cmpfunc);

    MPI_Bcast(&all_numbers, size, MPI_INT, root, MPI_COMM_WORLD);

    for (int i = 0; i < size; i++) {
        if (all_numbers[i] == x) {
            result = i;
            break;
        }
    }
    return result;
}
```

(a) Füllen Sie in den nachfolgenden Tabellen aus, wie die jeweils angegebenen Variablen nach Ausführung der in den Tabellen gegebenen Quelltext-Zeile belegt sind. Markieren Sie ggf. unbekannte Werte von Speicherstellen mit einem –. Nehmen Sie an, dass das Programm mit 4 Prozessen bearbeitet wird und die Methode je Prozess mit einem anderen Wert für x aufgerufen wird, aber demselben Wert für `root`: [4 Punkte]

```
Prozess 0: x = 3, root = 0
Prozess 1: x = 1, root = 0
Prozess 2: x = 7, root = 0
Prozess 3: x = 0, root = 0
```

(a) **Variable: `all_numbers`**

Prozess Nr.	**Wert nach Zeile 14 (Lösung)**
0	3, 1, 7, 0
1	-
2	-
3	-

(b) **Variable: `all_numbers`**

Prozess Nr.	**Wert nach Zeile 16 (Lösung)**
0	0, 1, 3, 7
1	-
2	-
3	-

(c) **Variable: `all_numbers`**

Prozess Nr.	**Wert nach Zeile 18 (Lösung)**
0	0, 1, 3, 7
1	0, 1, 3, 7
2	0, 1, 3, 7
3	0, 1, 3, 7

(d) **Variable: `result`**

Prozess Nr.	**Wert nach Zeile 25 (Lösung)**
0	2
1	1
2	3
3	0

Beispiellösung: Siehe Tabellen.

(b) Beschreiben Sie, welche Funktionalität von `foo` im Allgemeinen realisiert wird. Reine Erläuterungen der Programmabfolge im konkreten Beispielfall sind keine gültige Lösung und werden nicht gewertet. [1 Punkt]

Beispiellösung: `foo` gibt für jeden Prozess den Index des eigenen Werts (x) in der aufsteigend sortierten Liste aller Werte zurück.

(c) Gibt es in oben vorliegendem Programm noch weiteres Parallelisierungspotenzial? Wenn ja, nennen Sie eine solche Stelle. [2 Punkte]

Beispiellösung: Ja, das Sortieren ist aktuell sequentiell - das könnte man parallelisieren.

(d) Würde das Programm weiterhin dieselbe Funktionalität erfüllen, wenn nicht nur für den Parameter `x`, sondern auch für `root` unterschiedliche Werte pro Prozess übergeben werden würden? Begründen Sie kurz. [2 Punkte]

Beispiellösung: Nein, da die kollektiven Operationen in allen Prozessen mit demselben Wert für `root` aufgerufen werden müssen. Sonst ist nicht klar, welcher Prozess das Ergebnis aggregiert bzw. versendet.

(e) Ist der Aufruf von `MPI_Barrier` notwendig für die korrekte Ausführung des Programms? Begründen Sie kurz. [1 Punkt]

Beispiellösung: Nein, es werden bereits alle Prozesse durch das nachfolgende Gather synchronisiert.

(f) Ersetzen Sie den Aufruf von `MPI_Bcast` in Zeile 18 durch blockierende Punkt-zu-Punkt-Kommunikation, sodass der `all_numbers`-Puffer danach denselben Inhalt hat. [5 Punkte]

Beispiellösung:

```
1  MPI_Status status;
2  if (rank == root) {
3      for (int i = 0; i < size; i++) {
4          if (i != root) {
5              MPI_Send(&all_numbers, size, MPI_INT, i, 0,
                   MPI_COMM_WORLD);
6          }
7      }
8  } else {
9      MPI_Recv(&all_numbers, size, MPI_INT, root, MPI_ANY_TAG,
           MPI_COMM_WORLD, &status);
10 }
```

alternativ statt Deklaration von `MPI_Status`: `MPI_STATUS_IGNORE`

Aufgabe 7 (Java Happens-before) [10 Punkte]

Gegeben ist untenstehende Klasse `CarDealership` mit einer `main`-Methode, die auf der ebenfalls unten gegebenen Klasse `CarSale` die Methoden `carSold` sowie `carReturned` aufruft. Die Methode `carSold` soll sowohl die Anzahl der verkauften Autos inkrementieren, als auch das neue Auto der Liste verkaufter Autos hinzufügen und die neue Anzahl verkaufter Autos ausgeben. Die Methode `carReturned` soll die Anzahl verkaufter Autos dekrementieren, das Auto jedoch nicht von der Liste verkaufter Autos entfernen.

```
1  public class CarDealership {
2      private static CarSale carSale = new CarSale();
3
4      public static void main(String[] args){
5          Car car = new Car();
6          carSale.carSold(car);
7          carSale.carReturned(car);
8          System.out.println("Sold Cars: " + carSale.getNumberOfSoldCars());
9      }
10 }
11
12 public class CarSale {
13     public int numberOfSoldCars = 0;
14     public List<Car> soldCars = new ArrayList<Car>();
15
16     public void carSold(Car car) {
17         new Thread(()-> { numberOfSoldCars += 1; }).start();
18         new Thread(()-> { soldCars.add(car); }).start();
19         new Thread(()-> { System.out.println(numberOfSoldCars); }).start();
20     }
21
22     public void carReturned(Car car) {
23         new Thread(()-> { numberOfSoldCars -= 1; }).start();
24     }
25
26     public int getNumberOfSoldCars() {
27         return numberOfSoldCars;
28     }
29 }
30
31 public class Car { }
```

(a) Die `carSold`-Methode gibt nicht immer die erwartete Anzahl an `soldCars` aus, selbst wenn die Threads in der Reihenfolge von oben nach unten ausgeführt werden. Wodurch entsteht dieser Effekt? Verwenden Sie für Ihre Erklärung den Begriff der Happens-Before-Beziehung. [2 Punkte]

Beispiellösung: Er entsteht, da keine Happens-Before Beziehung vorliegt, wodurch memory inconsistencies möglich sind.

(b) Was müssen Sie am oben gezeigten Programm ändern, damit die Ausgabe der `carSold`-Methode immer wie erwartet funktioniert? Verändern Sie dabei nicht, dass jede Anweisung als eigener Thread gestartet wird. [1 Punkt]

Beispiellösung: Auf den beiden Threads die `join`-Methode aufrufen, bevor die Ausgabe gemacht wird. So wird eine Happens-Before Beziehung zwischen den Anweisungen etabliert.

(c) Ist es sicher, die Methoden `carSold` und `carReturned` in zwei Threads gleichzeitig aufzurufen? Begründen Sie kurz. Wenn nicht, wie könnten Sie dies sicherstellen? [3 Punkte]

Beispiellösung: Nein, hier kommt es zu einer Race Condition, da beide Methoden schreibend auf `numberOfSoldCars` zugreifen. Sicherstellen zB durch Verwenden von AtomicInteger statt int.

(d) Handelt es sich bei der `carSold`-Methode um Daten- oder Taskparallelismus? Begründen Sie Ihre Antwort. [2 Punkte]

Beispiellösung: Taskparallelismus, es sind zwei verschiedene Aufgaben.

(e) Ist es aus Performance-Perspektive sinnvoll, wie im Beispielprogramm jede Anweisung als eigenen Thread zu starten? Begründen Sie Ihre Antwort. [2 Punkte]

Beispiellösung: Nein, zu viel Overhead durch Starten der Threads, Koordinieren etc.

Aufgabe 8 (Rekursiver Abstieg) [18 Punkte]

In dieser Aufgabe sollen Sie nach der folgenden Grammatik einen rekursiven Abstiegsparser für λ-Kalkül-Typen entwickeln:

$$
\begin{array}{rcl}
\mathit{Type} & \rightarrow & \texttt{ident}\ \mathit{TypeRest} \\
 & | & \forall\ \texttt{ident}\ \texttt{.}\ \mathit{Type} \\
 & | & \texttt{(}\ \mathit{Type}\ \texttt{)}\ \mathit{TypeRest} \\
\mathit{TypeRest} & \rightarrow & \texttt{->}\ \mathit{Type} \\
 & | & \varepsilon
\end{array}
$$

Die Typsprache unterstützt Typvariablen, Funktionstypen, polymorphe Typen, und Klammern um die Präzedenz festzulegen.

(a) Geben Sie eine Klassenhierarchie für abstrakte Syntaxbäume von Typen an. Wählen Sie sprechende Namen für die Felder oder kommentieren Sie sie kurz(!). [5 Punkte]

Sie brauchen keine Accessor-Methoden und Konstruktoren anzugeben. Wir nehmen an, dass jede Klasse einen Konstruktor hat, der die Felder in ihrer Deklarationsreihenfolge zuweist. Zum Beispiel hätte die Klasse
`class C { String a; int b; }`
implizit einen Konstruktor
`C (String a, int b) { this.a = a; this.b = b; }`

Bewertungshinweis:
Für überflüssige Klassen werden Punkte abgezogen.

Beispiellösung:

```
abstract class Type { }

class Var extends Type {
    String name;
}

class Function extends Type {
    Type argument;
    Type result;
}

class Polymorph extends Type {
    Var boundVar; // oder "String boundVar;", *aber nicht* "Type boundType;"
    Type body;
}

// *keine* class Parentheses { ... }, class TypeRest { ... }
```

(b) Geben Sie für jedes Nichtterminal der Grammatik einen rekursiven Abstiegsparser an, der einen AST mithilfe Ihrer Klassenhierarchie aus (a) aufbaut. Brechen Sie bei Parserfehlern mit der globalen `error()`-Funktion ab. [13 Punkte]

Hinweis: Sie können davon ausgehen, dass die Lexer-Schnittstelle wie folgt aussieht:

- Die Token sind Werte eines `enum` namens **`Token`** und heißen **`LP`**, **`RP`**, **`ARROW`**, **`FORALL`**, **`DOT`**, **`IDENT`** und **`EOF`** für `(`, `)`, `->`, ∀, `.`, Identifier, sowie das Ende der Eingabe. Token können bspw. mit `==` verglichen werden.
- `lexer.current` gibt nichtkonsumierenden Zugriff auf das aktuelle Token.

- `lexer.lex()` konsumiert das aktuelle Token.
- `lexer.ident` enthält den zu einem **`IDENT`** gehörenden Namen als String.

```
String expect(Token tok) {
  if (lexer.current != tok) { error(); }
  String s = lexer.ident;
  lexer.lex();
  return s;
}
```

Beispiellösung:

```
    Type parseType() {
        switch (lexer.current) {
        case IDENT:
            Type var = new Var(expect(IDENT));
            return parseTypeRest(var);

        case FORALL:
            expect(FORALL);
            Var bound = new Var(expect(IDENT));
            expect(DOT);
            Type body = parseType();
            return new Polymorph(bound, body);

        case LP:
            expect(LP);
            Type t = parseType();
            expect(RP);
            return parseTypeRest(t);

        default:
            error();
            return null;
        }
    }

    Type parseTypeRest(Type left) {
        switch (lexer.current) {
        case ARROW:
            expect(ARROW);
            Type right = parseType();
            return new Function(left, right);

        case RP:
        case EOF:
            return left;

        default:
            error();
            return null;
        }
    }
```

Aufgabe 9 (Java-Bytecode: Heron) [8 Punkte]

Mit der folgenden Iteration kann man die Quadratwurzel von a annähern:

$$x_{n+1} = \frac{x_n + \frac{a}{x_n}}{2}$$

Wenn man $x_0 = a$ setzt, kann man abbrechen, sobald $x_n^2 \leq a$. Der Einfachheit halber rechnen wir hier mit ganzen Zahlen. Um Termination dieses Prozesses müssen Sie sich keine Gedanken machen.

(a) Übersetzen Sie die Iterationsvorschrift in umgekehrte polnische Notation. [2 Punkte]

(b) Implementieren Sie die Iteration in Java-Bytecode. Sämtliche Arithmetik soll auf Ganzzahlen stattfinden. Das (ganzzahlige) Argument a liegt in der lokalen Variable Nr. 1. In der lokalen Variable Nr. 2 soll am Ende Ihres Codes das Ergebnis stehen. Die weiteren lokalen Variablen dürfen Sie frei verwenden. [5 Punkte]

(c) Welche Java-Schleife entspricht dem Kontrollfluss Ihres Bytecode-Programms? Es reicht, wenn Sie den Schleifenkopf angeben, den Rumpf können Sie weglassen. [1 Punkt]

Beispiellösung:

(a) $x_n\ a\ x_n\ /\ +\ 2\ /$

(b)

```
      iload 1
      istore 2
loop: iload 2
      iload 1
      iload 2
      idiv
      iadd
      iconst_2
      idiv
      istore 2
      iload 2
      iload 2
      imul
      iload 1
      if_icmpgt loop
```

(c) Abhängig von der Implementierung von (b) z. B.

- `do { ... } while (xn * xn > a)` (entspricht der Lösung oben)
- `while (xn * xn > a) { ... }`